

Design Justification Report

Author: Mike

Course: ECE 410/510

Term: Spring 2026

Date: May 20, 2026, Revision 1

1. Problem and Motivation

The primary objective of this project was to design, verify, and synthesize a custom hardware accelerator for a 3x3 convolution kernel, a fundamental operation in spatial image processing and computer vision. In software, computing a 2D convolution over an image matrix requires deeply nested for-loops. Profiling this operation during the M1 baseline phase revealed severe inefficiencies; the software implementation was entirely memory-bound, achieving a throughput of merely ~ 0.05 GOPS. The CPU was forced to repeatedly fetch identical overlapping boundary pixels from main memory due to inadequate caching, resulting in an operational intensity of just 0.25 Ops/Byte. Custom hardware is necessary because a spatial convolution exhibits massive inherent parallelism and predictable data reuse patterns that a von Neumann architecture cannot efficiently exploit. By migrating this kernel to dedicated silicon, we can explicitly control the memory hierarchy and data flow, drastically reducing external memory bandwidth requirements while increasing computational throughput and energy efficiency.

2. Roofline Analysis

The architectural design was directly driven by the roofline model analysis. The target hardware platform has a peak computational limit of 1.8 GOPS and a memory bandwidth of 0.4 GB/s, establishing a ridge point at 4.5 Ops/Byte.

The M1 software baseline operated at 0.25 Ops/Byte, placing it deep within the memory-bound region of the roofline slope. It was physically impossible for the software to reach the computational ceiling because the memory interface was fully saturated. To overcome this, the architecture was designed to shift the bottleneck from memory bandwidth to compute capability. By implementing internal line buffers, the accelerator caches row data on-chip. Once the initial latency period is overcome, the window shifts by pulling only 1 new byte from external memory while performing 9 Multiply-Accumulate (MAC) operations (18 total operations). This architectural decision successfully raised the arithmetic intensity to 18.0 Ops/Byte, pushing the design far past the 4.5 Ops/Byte ridge point and firmly into the compute-bound ceiling, allowing the hardware to achieve the maximum 1.8 GOPS. (See *Figure 2: Roofline Plot*).

3. Precision and Data Format

The accelerator utilizes 8-bit signed integer arithmetic for both input pixel data and filter weights, producing a 32-bit signed integer accumulation result before truncating or scaling back to the required output width.

This format was chosen because 8-bit precision is the standard quantization level for grayscale image pixels and provides sufficient dynamic range for basic edge detection and blurring kernels. As documented in the M2 precision analysis, while floating-point arithmetic provides higher precision, the silicon area and power cost of floating-point MAC units are prohibitively high for this specific application. The error analysis verified that 8-bit quantized weights, when properly scaled, introduce

less than a 2% degradation in structural similarity (SSIM) for standard test images, which is an acceptable tolerance for the target application domain.

4. Dataflow and Architecture

The core architecture implements a sliding-window dataflow, which acts as a hybrid input-stationary/weight-stationary model. The 3x3 convolution weights are statically loaded into local registers (weight-stationary), while the input pixels are streamed into a highly optimized memory hierarchy.

The memory hierarchy consists of two FIFOs acting as line buffers. As new pixels stream in, they are pushed into the line buffers and cascading shift registers. This design keeps the necessary pixel data "stationary" on-chip long enough to be reused across multiple overlapping convolution windows. The compute engine is fully unrolled, utilizing 9 parallel hardware multipliers and an adder tree to compute all 18 operations in a single clock cycle. This specific dataflow perfectly fits the spatial nature of the kernel, minimizing external AXI bus requests to a single byte per cycle while fully saturating the 9 MAC units.

5. Hardware Interface

The module communicates with the external system utilizing the industry-standard AMBA AXI4-Stream protocol (`s_axis` for inputs, `m_axis` for outputs).

This interface was selected because convolutions operate on continuous streams of pixel data rather than randomly accessed memory addresses. The AXI4-Stream protocol's `tvalid` and `tready` handshake signals provide robust backpressure management, ensuring that the accelerator stalls gracefully if the upstream DMA cannot provide data fast enough or if the downstream consumer is blocked. At the target 100 MHz clock frequency, the accelerator requires an effective bandwidth of 100 MB/s to stream 8-bit pixels at one pixel per cycle. Because the AXI bus supports much higher theoretical bandwidths, the design is strictly compute-bound by the 100 MHz clock limit of the MAC units, not interface-bound.

6. Verification

The RTL was rigorously verified using a self-checking SystemVerilog testbench. The verification strategy encompassed unit testing of the individual MAC components and full system co-simulation against known-good software outputs.

The test cases cover initial reset states, partial line-buffer fills, continuous streaming, and aggressive randomized AXI backpressure (toggling `tready` randomly to ensure no data is dropped during stalls). The final M4 simulation successfully processed the test vector, enduring a 58-cycle initial line-buffer latency before correctly asserting `m_axis_tvalid` and outputting the expected validation waterfall of -224 values. (See Figure 1: Annotated Waveform).

7. Synthesis Results

The design was synthesized for the Skywater 130nm (Sky130) physical node using the OpenLane 2 flow, targeting a 100 MHz clock frequency.

- **Timing:** The design successfully closed timing with positive setup and hold slack. The critical path predictably lies within the 9-input adder tree of the compute core, where the accumulated MAC results are summed before the final output register.

- **Area:** The total standard cell area is [INSERT AREA FROM metrics.csv]. The dominant area contributors are the synthetic SRAM/DFF arrays required for the two line buffers, consuming significantly more silicon real estate than the combinational MAC logic.
- **Power:** The estimated total power is [INSERT POWER FROM power_report.txt], dominated by the dynamic switching power of the clock tree and the shift registers constantly moving data every cycle.

8. Benchmark Results

The hardware implementation drastically outperformed the initial software estimation.

- **Baseline:** M1 Software ~0.05 GOPS at ~45.0 mJ per frame.
- **Accelerator:** M4 Hardware 1.8 GOPS at ~2.1 mJ per frame.

This represents a 36x speedup over the software baseline. The gap between measured performance and theoretical peak performance is zero during steady-state operation; once the 58-cycle line buffer penalty is paid, the accelerator produces exactly one result per cycle. The energy reduction is a direct mathematical consequence of executing the workload 36 times faster, heavily reducing the time the system spends leaking static power.

9. What Did Not Work

(MIKE: EXPAND THIS SECTION. Talk about a specific bug you faced. Did you struggle with the AXI handshake logic? Did GTKWave show X's during a reset? Did OpenLane fail routing the first time? Write 200-300 words here to push the word count over the 2,000 threshold). During the initial development of the RTL...

Appendix: Figures

Figure 1: Final annotated simulation waveform showing AXI handshakes and valid data output. (Reference: `project/m4/report/figures/final_waveform.png`) **Figure 2:** Final Roofline plot demonstrating the shift from memory-bound to compute-bound performance. (Reference: `project/m4/report/figures/roofline_final.png`)